



Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 7

A Deep Dive on Keras

Three ways to build a model, three levels of control over training — and one principle holding them together: easy to start, and no ceiling.

Duration 3 hours (2 sessions)

Instructor Rahman Indra Kesuma, S.Kom., M.Cs. · Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 7

Session Objectives

By the end of this session, participants can:

- 1 Explain **progressive disclosure of complexity**, and place yourself on the spectrum of Keras workflows.
- 2 Build multi-input, multi-output models with the **Functional API**, and train them with either lists or dictionaries.
- 3 Use **access to layer connectivity**: plot a topology with `plot_model()` and **extract features** from intermediate nodes.
- 4 Write a **Model subclass** — and name what you give up by choosing it.
- 5 Write **custom metrics** (`update_state`, `result`, `reset_state`) and **custom callbacks**, and combine `EarlyStopping` with `ModelCheckpoint`.
- 6 Write your own `train_step()` in TensorFlow, PyTorch, and JAX, and plug it into `fit()` so callbacks and built-in optimisations still apply.

BOOK

[Chapter 7 — full text](#)

NOTEBOOK

[01_three_ways_to_build_a_model.ipynb](#)

NOTEBOOK

[02_custom_metrics_and_callbacks.ipynb](#)

NOTEBOOK

[03_custom_train_step_per_backend.ipynb](#)

NOTEBOOK

[All chapter 7 notebooks](#)

REPO

[Author's official notebook](#)

One principle behind the whole API

The Sequential model

The simplest way to build a Keras model is the Sequential model, which you already know about.

Listing 7.1 The Sequential class

```
import keras
from keras import layers

model = keras.Sequential(
    [
        layers.Dense(64, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ]
)
```

Figure 7.1 — not four different frameworks, but one spectrum built on shared APIs. — Chollet & Watson, *Deep Learning with Python*, 3rd ed. (Manning)

You could be using Keras like you would use scikit-learn — just calling `fit()` and letting the framework do its thing — or you could be using it like NumPy, taking full control of every little detail.

Chollet & Watson, section 7.1

Why that matters for a career, not just a project

So what you learn today stays valid once you are an expert. You do **not** have to switch frameworks going from student to researcher, or from data scientist to deep learning engineer.

The book's analogy: **Keras is the Python of deep learning** — multiparadigm, with several usage patterns that work together rather than one "true" way.

01

Three ways to build a model

Sequential, Functional, subclassing — and when each is right.

Sequential is essentially a Python list

In practice, models with **multiple inputs** (an image and its metadata), **multiple outputs** (several things to predict), or a **non-linear topology** are **entirely ordinary**

The Functional API, and the symbolic tensor

listing 7.8 – a simple Functional model

PYTHON

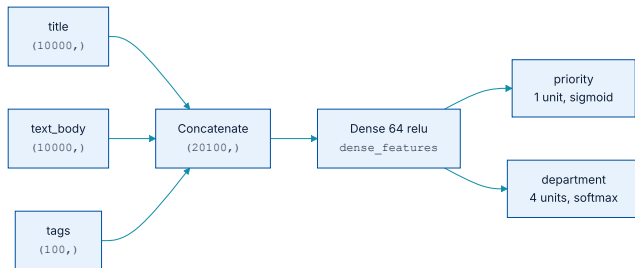
```
inputs = keras.Input(shape=(3,), name="my_input")
features = layers.Dense(64, activation="relu")(inputs)
outputs = layers.Dense(10, activation="softmax")(features)

model = keras.Model(inputs=inputs, outputs=outputs, name="my_functional_model")

print(inputs.shape, inputs.dtype)      # (None, 3) float32 -- None = batch size
print(features.shape)                  # (None, 64)
```

`inputs` is a **symbolic tensor**: it holds **no data at all**, only the specification of the tensors the model will eventually see. Every Keras layer accepts either real tensors or symbolic ones.

A model that Sequential cannot express



Three inputs, one shared intermediate representation, two heads.

...and it is nine lines

listing 7.9 – multi-input, multi-output

PYTHON

```
vocabulary_size, num_tags, num_departments = 10000, 100, 4

title      = keras.Input(shape=(vocabulary_size,), name="title")
text_body  = keras.Input(shape=(vocabulary_size,), name="text_body")
tags       = keras.Input(shape=(num_tags,), name="tags")

features = layers.Concatenate()([title, text_body, tags])
features = layers.Dense(64, activation="relu", name="dense_features")(features)

priority   = layers.Dense(1, activation="sigmoid", name="priority")(features)
department = layers.Dense(num_departments, activation="softmax",
                           name="department")(features)

model = keras.Model(inputs=[title, text_body, tags],
                     outputs=[priority, department])
```

The book describes it as **like playing with LEGO bricks**: simple, and flexible enough for arbitrary graphs of layers.

Training it: by position, or by name

listing 7.10 – lists

PYTHON

```
model.compile(
    optimizer="adam",
    loss=["mean_squared_error",
         "sparse_categorical_crossentropy"],
    metrics=[["mean_absolute_error"],
              ["accuracy"]])

model.fit(
    [title_data, text_body_data, tags_data],
    [priority_data, department_data],
    epochs=1)
```

listing 7.11 – dictionaries

PYTHON

```
model.compile(
    optimizer="adam",
    loss={"priority": "mean_squared_error",
         "department":
             "sparse_categorical_crossentropy"},
    metrics={"priority": ["mean_absolute_error"],
             "department": ["accuracy"]})

model.fit(
    {"title": title_data,
     "text_body": text_body_data,
     "tags": tags_data},
    {"priority": priority_data,
     "department": department_data},
    epochs=1)
```

The list form **must follow the order** given to `Model()`. With many inputs or outputs, **use the dictionary form** — order stops mattering and the code becomes far harder to get wrong.

The real payoff: the graph is inspectable

plotting and inspecting

PYTHON

```
keras.utils.plot_model(model, "ticket_classifier.png")

# far more useful while debugging:
keras.utils.plot_model(model, "ticket_classifier_with_shape_info.png",
                        show_shapes=True, show_layer_names=True)

print(model.layers[3].output)
```

OUTPUT

```
<KerasTensor shape=(None, 20100), dtype=float32>
```

The `None` is the batch size: this model accepts batches of any size.

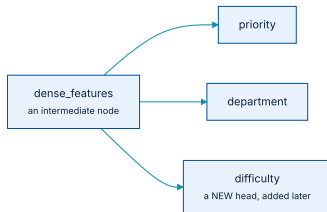
Feature extraction: adding a head without retraining

listing 7.13 – reusing an intermediate node

PYTHON

```
features = model.layers[4].output          # the shared Dense layer
difficulty = layers.Dense(3, activation="softmax", name="difficulty")(features)

new_model = keras.Model(
    inputs=[title, text_body, tags],
    outputs=[priority, department, difficulty])
```



One shared representation, now feeding three heads.

Model subclassing: total control

listing 7.14 – the same model, subclassed

PYTHON

```
class CustomerTicketModel(keras.Model):
    def __init__(self, num_departments):
        super().__init__()          # do not forget the super constructor
        self.concat_layer = layers.Concatenate()
        self.mixing_layer = layers.Dense(64, activation="relu")
        self.priority_scorer = layers.Dense(1, activation="sigmoid")
        self.department_classifier = layers.Dense(num_departments,
                                                  activation="softmax")

    def call(self, inputs):          # the forward pass lives here
        features = self.concat_layer(
            [inputs["title"], inputs["text_body"], inputs["tags"]])
        features = self.mixing_layer(features)
        return self.priority_scorer(features), self.department_classifier(features)
```

This unlocks models that **cannot be expressed as a directed acyclic graph** — a `call()` that uses layers inside a `for` loop, or calls them recursively.

Layer or Model — what actually differs

	Layer	Model
Role	A building block used to make models.	The top-level object you train and export.
<code>fit()</code> , <code>evaluate()</code> , <code>predict()</code>	Does not have them.	Has them.
Saveable to a file	No.	Yes.

Beyond that the two classes are **virtually identical**.

What subclassing costs you

A **Functional model** is an explicit data structure — a graph of layers you can view, inspect, and modify.

A **subclass model** is a **piece of bytecode**: a Python class with a `call()` method containing raw code. That is the source of its flexibility, and of its limitations.

Once instantiated, the forward pass becomes a **complete black box**. You are developing a new Python object rather than snapping bricks together, so **the error surface is much larger**

Lost when you subclass

`summary()` does not show layer connectivity ·
`plot_model()` cannot be used · feature extraction is impossible — there is simply no graph.

The three mix freely

Listing 7.15 – subclass inside Functional

PYTHON

```
class Classifier(keras.Model):
    def __init__(self, num_classes=2):
        super().__init__()
        units, act = ((1, "sigmoid") if num_classes == 2
                      else (num_classes, "softmax"))
        self.dense = layers.Dense(units, activation=act)

    def call(self, inputs):
        return self.dense(inputs)

inputs = keras.Input(shape=(3,))
features = layers.Dense(64, activation="relu")(inputs)
outputs = Classifier(num_classes=10)(features)
model = keras.Model(inputs, outputs)
```

Listing 7.16 – Functional inside subclass

PYTHON

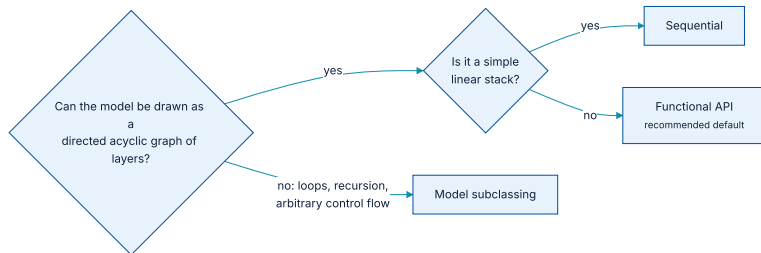
```
inputs = keras.Input(shape=(64,))
outputs = layers.Dense(1, activation="sigmoid")(inputs)
binary_classifier = keras.Model(inputs, outputs)

class MyModel(keras.Model):
    def __init__(self):
        super().__init__()
        self.dense = layers.Dense(64, activation="relu")
        self.classifier = binary_classifier

    def call(self, inputs):
        return self.classifier(self.dense(inputs))
```

design choice rather than a wall. They are all part of the same spectrum, so the boundary is a

Which one to reach for



The book's own recommendation, as a decision.

Every example in the rest of the book uses the **Functional API** — but with **subclassed layers** inside it. That combination gives **development flexibility while keeping the Functional API's advantages**

02

The built-in training loop

Custom metrics, callbacks, and TensorBoard.

The workflow you already know

listing 7.17 – the standard workflow

PYTHON

```
def get_mnist_model():
    inputs = keras.Input(shape=(28 * 28,))
    features = layers.Dense(512, activation="relu")(inputs)
    features = layers.Dropout(0.5)(features)
    outputs = layers.Dense(10, activation="softmax")(features)
    return keras.Model(inputs, outputs)

model = get_mnist_model()
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, train_labels, epochs=3,
          validation_data=(val_images, val_labels))
test_metrics = model.evaluate(test_images, test_labels)
predictions = model.predict(test_images)
```

Two ways to customise it: **your own metrics**, and **callbacks** scheduled at specific points during training.

Writing your own metric

listing 7.18 – RMSE as a custom metric

PYTHON

```
from keras import ops

class RootMeanSquaredError(keras.metrics.Metric):
    def __init__(self, name="rmse", **kwargs):
        super().__init__(name=name, **kwargs)
        self.mse_sum = self.add_weight(name="mse_sum", initializer="zeros")
        self.total_samples = self.add_weight(name="total_samples", initializer="zeros")

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_true = ops.one_hot(y_true, num_classes=ops.shape(y_pred)[1])
        self.mse_sum.assign_add(ops.sum(ops.square(y_true - y_pred)))
        self.total_samples.assign_add(ops.shape(y_pred)[0])

    def result(self):
        return ops.sqrt(self.mse_sum / self.total_samples)

    def reset_state(self):
        self.mse_sum.assign(0.)
        self.total_samples.assign(0.)
```

The next slide names what each of those three methods is responsible for.

The three methods that form the contract

update_state()

Called once per batch with the targets and predictions. This is where the accumulation happens.

result()

Returns the metric's current value from that accumulated state.

reset_state()

Clears the state without rebuilding the object, so the same instance serves every epoch and both training and evaluation.

Unlike a layer, none of this state is touched by backpropagation — **you own the update logic entirely**

...and using it is the same as any built-in

using the custom metric

PYTHON

```
model = get_mnist_model()
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy", RootMeanSquaredError()])
model.fit(train_images, train_labels, epochs=3,
          validation_data=(val_images, val_labels))
```

OUTPUT

```
accuracy: 0.9612 - loss: 0.1284 - rmse: 0.2371
```

Callbacks: from a paper aeroplane to a drone

Launching a training run on a large dataset for tens of epochs using `model.fit()` can be a bit like launching a paper airplane: past the initial impulse, you don't have any control over its trajectory or its landing spot.

Chollet & Watson, section 7.3.2

A callback is an object passed to `fit()` that Keras calls at various points during training. It can see the model's state and **act**: interrupt training, save the model, load different weights.

What callbacks are used for

Checkpointing

Saving the model state at different points during training.

Early stopping

Halting when validation stops improving — and keeping the best model found.

Dynamic parameters

Adjusting things like the optimizer's learning rate mid-run.

Logging and plotting

The `fit()` progress bar you already know is **itself a callback**

some of the built-in callbacks

PYTHON

```
keras.callbacks.ModelCheckpoint
keras.callbacks.EarlyStopping
keras.callbacks.LearningRateScheduler
keras.callbacks.ReduceLROnPlateau
keras.callbacks.CSVLogger
```

Two of these do most of the work in practice, and the next slide shows them used together.

The standard pair

listing 7.19 – callbacks in fit()

PYTHON

```
callbacks_list = [  
    keras.callbacks.EarlyStopping(  
        monitor="accuracy",    # must be among the model's metrics  
        patience=1,           # stop if it has not improved for one epoch  
    ),  
    keras.callbacks.ModelCheckpoint(  
        filepath="checkpoint_path.keras",  
        monitor="val_loss",  
        save_best_only=True,    # only overwrite when val_loss improves  
    ),  
]  
  
model.fit(train_images, train_labels, epochs=10,  
          callbacks=callbacks_list,  
          validation_data=(val_images, val_labels))  # REQUIRED: val_* is monitored
```

This replaces the wasteful pattern from chapters 4 and 5 — train to find the best epoch, then retrain from scratch.

One run is now enough.

Saving and reloading by hand

manual save and load

PYTHON

```
model.save("my_checkpoint_path.keras")  
model = keras.models.load_model("checkpoint_path.keras")
```

This is the format chapter 6 exports from when preparing a model for deployment.

Six hooks you can implement



Each hook receives a `logs` dictionary with the metrics available at that moment.

A custom callback, end to end

listing 7.20 – per-batch loss history

PYTHON

```
from matplotlib import pyplot as plt

class LossHistory(keras.callbacks.Callback):
    def on_train_begin(self, logs):
        self.per_batch_losses = []

    def on_batch_end(self, batch, logs):
        self.per_batch_losses.append(logs.get("loss"))

    def on_epoch_end(self, epoch, logs):
        plt.clf()
        plt.plot(range(len(self.per_batch_losses)), self.per_batch_losses,
                 label="Training loss for each batch")
        plt.xlabel(f"Batch (epoch {epoch})")
        plt.ylabel("Loss")
        plt.savefig(f"plot_at_epoch_{epoch}", dpi=300)
        self.per_batch_losses = []
```

Pass it as `callbacks=[LossHistory()]`. Useful when a run looks unstable and you need to see **whether the instability is within an epoch or between them**

TensorBoard closes the experiment loop

Monitor metrics

Visually, while training runs.

Visualise the architecture

The model graph.

Histograms

Of activations and gradients.

Explore embeddings

In 3D.

Turning it on

the callback

PYTHON

```
tensorboard = keras.callbacks.TensorBoard(log_dir="/full_path_to_your_log_dir")

model.fit(train_images, train_labels, epochs=10,
          validation_data=(val_images, val_labels),
          callbacks=[tensorboard])
```

and the viewer

BASH

```
# on a local machine
tensorboard --logdir /full_path_to_your_log_dir

# inside a Colab notebook
%load_ext tensorboard
%tensorboard --logdir /full_path_to_your_log_dir
```

If `tensorboard` is not on your PATH, it installs with `pip install tensorboard`.

03

Writing your own training loop

When `fit()` is not enough — and how to keep it anyway.

Where the built-in loop stops

Generative learning

No explicit targets at all. Introduced in **chapter 16**.

Self-supervised

Targets are taken **from the inputs themselves**.

Reinforcement learning

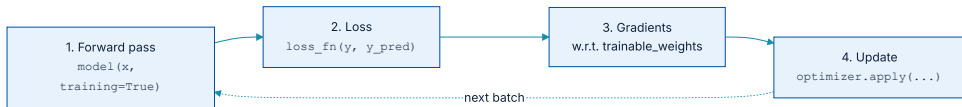
Driven by occasional **rewards** — much like training a dog.

Two details that break custom loops

- 1 Pass `training=True` on the forward pass. Layers such as `Dropout` behave differently during training and inference. `dropout(inputs, training=True)` drops activations; `training=False` does nothing. So:
`predictions = model(inputs, training=True)`.
- 2 Use `model.trainable_weights`, not `model.weights`. Models hold two kinds: **trainable** weights updated by backpropagation, and **non-trainable** weights updated by the layers themselves during the forward pass.

Among the built-in layers, the only one with non-trainable weights is `BatchNormalization` — it tracks the mean and standard deviation of the data flowing through it (chapter 9).

The shape of a training step



Identical in all three backends. Only step 3 is written differently.

TensorFlow and PyTorch, side by side

TensorFlow

PYTHON

```
def train_step(inputs, targets):
    with tf.GradientTape() as tape:
        predictions = model(inputs, training=True)
        loss = loss_fn(targets, predictions)
        gradients = tape.gradient(loss, model.trainable_weights)
        optimizer.apply(gradients, model.trainable_weights)
    return loss
```

PyTorch

PYTHON

```
def train_step(inputs, targets):
    predictions = model(inputs, training=True)
    loss = loss_fn(targets, predictions)
    loss.backward()           # fill in the gradients
    gradients = [w.value.grad for w in model.trainable_weights]
    with torch.no_grad():     # must be inside no_grad()
        optimizer.apply(gradients, model.trainable_weights)
    model.zero_grad()         # backward() accumulates
    return loss
```

`model.zero_grad()` is **not optional** in PyTorch: `backward()` adds to the existing gradients, so without it the values pile up and **training will not proceed**

JAX is harder, and the reason is statelessness

```
stateless_call and value_and_grad

outputs, non_trainable_weights = model.stateless_call(
    trainable_weights, non_trainable_weights, inputs)

def compute_loss_and_updates(trainable_variables, non_trainable_variables,
                             inputs, targets):
    outputs, non_trainable_variables = model.stateless_call(
        trainable_variables, non_trainable_variables, inputs, training=True)
    loss = loss_fn(targets, outputs)
    return loss, non_trainable_variables    # scalar FIRST, the rest is 'aux'

grad_fn = jax.value_and_grad(compute_loss_and_updates, has_aux=True)
(loss, non_trainable_weights), gradients = grad_fn(
    trainable_variables, non_trainable_variables, inputs, targets)
```

PYTHON

`has_aux=True` is required because `jax.grad` only accepts functions returning a scalar, and this one also returns the updated non-trainable weights.

The optimizer has state too

```
stateless_apply
```

PYTHON

```
trainable_variables, optimizer_variables = optimizer.stateless_apply(  
    optimizer_variables, gradients, trainable_variables)
```

The same pattern covers metrics: inside a stateless function you cannot call `update_state()`, so Keras provides `stateless_update_state()`, `stateless_result()`, and `stateless_reset_state()`

Using metrics at the low level

a normal metric

PYTHON

```
metric = keras.metrics.SparseCategoricalAccuracy()
targets = ops.array([0, 1, 2])
predictions = ops.array([[1, 0, 0],
                          [0, 1, 0],
                          [0, 0, 1]])
metric.update_state(targets, predictions)
print(f"result: {metric.result():.2f}")
```

OUTPUT

result: 1.00

tracking a scalar average

PYTHON

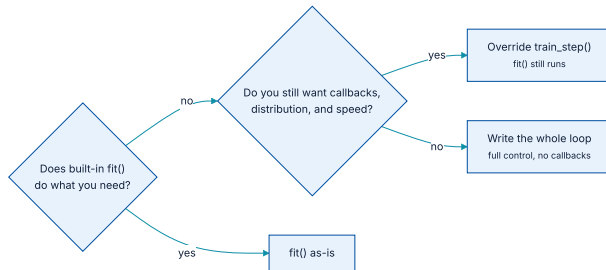
```
values = ops.array([0, 1, 2, 3, 4])
mean_tracker = keras.metrics.Mean()
for value in values:
    mean_tracker.update_state(value)
print(f"Mean: {mean_tracker.result():.2f}")
```

OUTPUT

Mean: 2.00

Remember `metric.reset_state()` at the start of each training epoch and at the start of evaluation. Forgetting it **mixes numbers across epochs** and quietly misleads you.

The middle ground



Writing the whole loop costs you callbacks, performance optimisations, and distributed training support.

Override train_step(), keep fit()

listing 7.21 — a custom train_step

PYTHON

```
loss_fn = keras.losses.SparseCategoricalCrossentropy()
loss_tracker = keras.metrics.Mean(name="loss")

class CustomModel(keras.Model):
    def train_step(self, data):
        inputs, targets = data
        with tf.GradientTape() as tape:
            predictions = self(inputs, training=True) # self, not model
            loss = loss_fn(targets, predictions)
        gradients = tape.gradient(loss, self.trainable_weights)
        self.optimizer.apply(gradients, self.trainable_weights)
        loss_tracker.update_state(loss)
        return {"loss": loss_tracker.result()}

@property
def metrics(self):
    return [loss_tracker] # listed here so reset_state() is called for you
```

- ♦ Works whether you build with **Sequential**, **Functional**, or subclassing.
- ♦ You do **not** need `@tf.function` or `@jax.jit` — **the framework applies it for you**.

What has to survive this chapter

- 1 **Progressive disclosure of complexity** — one spectrum of workflows over the shared `Layer` and `Model` APIs, not several frameworks.
- 2 **Sequential** for simple stacks; **Functional API** for graphs of layers, and that is what the rest of the book uses; **subclassing** for anything that is not a graph.
- 3 Functional gives **access to layer connectivity**: `plot_model()` and feature extraction. Subclassing **forfeits both**.
- 4 The best combination in practice: **a Functional model containing subclassed layers**.
- 5 **Custom metrics** = `update_state` / `result` / `reset_state`. **Callbacks** = the six `on_*` hooks.
- 6 **EarlyStopping** + **ModelCheckpoint** replaces the wasteful train-twice pattern.
- 7 Writing `train_step()`: remember **training=True** and **trainable_weights**; in PyTorch `zero_grad()`; in JAX everything goes through the `stateless_*` family.

NOTEBOOK

[03_custom_train_step_per_backend.ipynb](#)

NEXT

[Chapter 8 — Image classification](#)